

Raport końcowy

Problem komiwojażera krótkodystansowca (Bottleneck TSP)
Algorytmy zaawansowane — projekt 2025/2026

Hubert Sobociński

Sandra Adamiec

Piotr Tyrakowski

21 maja 2026

Streszczenie

Niniejszy raport podsumowuje implementację 3-aproksymacji problemu Bottleneck TSP: algorytm Prima budujący MST oraz rekurencyjną konstrukcję cyklu Hamiltona wykorzystującą własność sześcianu drzewa (T^3). Przedstawiono instrukcję uruchomienia programu, wyniki testów poprawności (`pytest`) oraz analizę eksperymentalną złożoności czasowej i jakości aproksymacji na instancjach euklidesowych. Empirycznie potwierdzono, że czas działania rośnie w sposób wielomianowy — pomiar w skali log-log dał nachylenie $\approx 1,64$, co jest zgodne z teoretyczną złożonością $O(n^2)$. Współczynnik aproksymacji w pomiarach pozostaje znacznie poniżej gwarancji teoretycznej 3 (typowo 1,0–1,3).

Spis treści

1	Podsumowanie projektu	3
1.1	Cel	3
1.2	Zakres implementacji	3
1.3	Autorzy i podział prac	3
2	Opis algorytmu (skrót)	3
3	Zgodność z dokumentacją wstępną i zmiany w implementacji	3
3.1	Elementy dodane po dokumentacji wstępnej	4
3.2	Różnica formatu wyjścia	4
3.3	Uwaga do dowodu we wstępnej	4
4	Instrukcja uruchomienia dla użytkownika	5
4.1	Wymagania	5
4.2	Szybki start — rozwiązanie jednej instancji	5
4.3	Generator danych testowych	5
4.4	Benchmark i wykresy (do raportu)	5
4.5	Testy automatyczne	5
5	Analiza eksperymentalna	5
5.1	Metodologia	5
5.2	Złożoność czasowa	6
5.3	Jakość aproksymacji (3-aproksymacja)	7
5.4	Porównanie czasów: aproksymacja vs brute force	8
5.5	Wizualizacja tras — porównanie z optimum	9
5.6	Wizualizacja MST oraz cyklu aproksymacji	10
6	Wnioski końcowe	10

7	Podsumowanie testów automatycznych	10
8	Bibliografia	10

1 Podsumowanie projektu

1.1 Cel

Celem projektu było zaprojektowanie i zaimplementowanie algorytmu aproksymacyjnego dla wariantu *Bottleneck TSP*, w którym minimalizuje się wagę **najcięższej** krawędzi na cyklu Hamiltona (a nie sumę wag jak w klasycznym TSP). Graf wejściowy jest pełny i spełnia nierówność trójkąta.

1.2 Zakres implementacji

Repozytorium <https://github.com/hubsonsss/TSP-Bottleneck> zawiera:

- `bottleneck_tsp.py` — solver 3-aproksymacji,
- `generator.py` — generator instancji testowych (macierze euklidesowe),
- `benchmark.py` — pomiary czasu i porównanie z brute force,
- `plots.py` — generowanie wykresów do raportu,
- `tests/` — 59 testów automatycznych (poprawność i wydajność).

1.3 Autorzy i podział prac

Autor	Zakres
Hubert Sobociński	Implementacja algorytmu, generator danych testowych, generator wykresów, raport końcowy
Sandra Adamiec	Testy automatyczne, benchmark, dokumentacja testów, raport końcowy, dokumentacja wstępna
Piotr Tyrakowski	Analiza teoretyczna, pseudokod, dokumentacja wstępna

Tabela 1: Podział prac w zespole.

2 Opis algorytmu (skrót)

Algorytm składa się z dwóch etapów:

1. **MST (Prim)** — minimalne drzewo rozpinające z macierzy wag w czasie $O(n^2)$.
2. **Cykl z T^3** — rekurencyjne sklejanie ścieżek Hamiltona; wirtualne skoki łączą wierzchołki odległe o co najwyżej trzy krawędzie w T , co przy nierówności trójkąta daje 3-aproksymację.

Złożoność całkowita dokumentowana w projekcie: $O(n^2)$ (dominuje Prim na grafie pełnym zapisanym macierzowo). Szczegółowy pseudokod, dowód poprawności i 3-aproksymacji pozostają w **dokumentacji wstępnej** (załącznik teoretyczny).

3 Zgodność z dokumentacją wstępną i zmiany w implementacji

Dokumentacja wstępna (14 kwietnia 2026) opisuje założenia teoretyczne; niniejszy raport końcowy oraz kod w repozytorium opisują **stan faktyczny** po zakończeniu implementacji i testów (21 maja 2026). Poniższa tabela zestawia zgodność oraz świadome różnice.

Element	Dokumentacja wstępna	Implementacja / raport końcowy
Cel problemu	Bottleneck TSP, min. najcięższa krawędź cyklu	Bez zmian
Założenia grafu	Pełny, symetryczny, nierówność trójkąta	Bez zmian; generator domyślnie tworzy instancje euklidesowe (spełniając założenia)
Algorytm	Prim (MST) + rekurencja T^3	Bez zmian w logice; <code>deque</code> zamiast list wiązanych z wstępnej (równoważne $O(1)$ przy sklejanju)
Złożoność	$O(n^2)$ całkowita	Bez zmian; empirycznie nachylenie $\log\text{--}\log \approx 1,64$, zgodne z $O(n^2)$ (sekcja 5)
Format wejścia	Macierz $n \times n$, $w_{ii} = 0$	Zgodny — <code>load_from_file</code>
Format wyjścia	CYCLE_LENGTH, MAX_EDGE_WEIGHT, wierzchołki $1 \dots n$	Inny : tekst polski, cykl z <code>-></code> , wierzchołki $0 \dots n-1$ (sekcja 3.2)
Pseudokod i dowód	Pełne w dokumencie wstępnym	Nie powtórzone w raporcie końcowym — odwołanie do wstępnej
Analiza jakości	Dowód $\leq 3 \cdot OPT$	Dodatkowo: testy na brute force, wykresy ratio (sekcja 5)

Tabela 2: Porównanie dokumentacji wstępnej z implementacją.

3.1 Elementy dodane po dokumentacji wstępnej

- `generator.py` — generowanie instancji testowych (tryb euklidesowy i wsadowy),
- `benchmark.py` oraz `plots.py` — pomiary czasu, porównanie z brute force, wykresy,
- `tests/` — 59 testów automatycznych (`pytest`),
- wizualizacja MST i cyklu (`-viz`),
- tryb konsolowy (`-console`) i parametr `-o` dla pliku wynikowego.

3.2 Różnica formatu wyjścia

Wstępna (przykład):

```
CYCLE_LENGTH 4
1 3 4 2 1
MAX_EDGE_WEIGHT 5
```

Program (`output.txt`):

```
Bottleneck (najcięższa krawędź): 9.0
Cykl Hamiltona:
0 -> 2 -> 4 -> 6 -> 8 -> 9 -> ...
```

Zmiany względem specyfikacji wstępnej:

- brak linii `CYCLE_LENGTH` — długość cyklu wynika z listy wierzchołków,
- **numeracja od 0** (implementacja macierzowa) zamiast od 1 z przykładu wstępnego,
- inna etykieta pola bottleneck (`Bottleneck` zamiast `MAX_EDGE_WEIGHT`).

Wartość wyniku (bottleneck) jest ta sama co we wstępnej; różni się wyłącznie *prezentacja*. Ujednolicenie formatu wyjścia do wzorca ze wstępnej pozostaje możliwą pracą porządkową (patrz wnioski końcowe).

3.3 Uwaga do dowodu we wstępnej

W dowodzie 3-aproksymacji dokumentacja wstępna odwołuje się do własności MST w kontekście algorytmu **Kruskala**, natomiast w kodzie zastosowano **Prima** na macierzy wag (zgodnie z opisem

etapu 1 we wstępnej). Oba algorytmy budują drzewo rozpinające; schemat aproksymacji przez T^3 pozostaje zgodny z założeniami projektu.

4 Instrukcja uruchomienia dla użytkownika

4.1 Wymagania

- Python 3.10+ (testowano na 3.12),
- numpy, matplotlib — solver z wizualizacją i benchmark,
- pytest — opcjonalnie, do testów (`pip install -r requirements-dev.txt`).

4.2 Szybki start — rozwiązanie jednej instancji

```
git clone https://github.com/hubsonsss/TSP-Bottleneck.git
cd TSP-Bottleneck
python bottleneck_tsp.py input.txt
python bottleneck_tsp.py input.txt -o wynik.txt --viz
```

Format pliku wejściowego (`input.txt`):

```
n
w_11 w_12 ... w_1n
...
w_n1 ... w_nn
```

Pierwsza liczba to n ; macierz symetryczna, $w_{ii} = 0$, wagi ≥ 0 , nierówność trójkąta.

Wyjście (konsola i plik): najcięższa krawędź cyklu oraz kolejność wierzchołków (cykl zamknięty).

4.3 Generator danych testowych

```
python generator.py 10 -o moja_instancja.txt --seed 42
python generator.py --batch --sizes 5,10,20,50,100,200 --repeats 5 --out-dir test_inputs --seed 42
```

4.4 Benchmark i wykresy (do raportu)

```
python benchmark.py --sizes 5,10,20,50,100,200 --repeats 5
python benchmark.py --skip-timing --skip-quality --comparison --max-n 10 --repeats 5
python plots.py --routes --route-sizes 6,8,10
```

Wyniki CSV: katalog `results/`; wykresy PNG: katalog `charts/`.

4.5 Testy automatyczne

```
pytest tests/ -v
# lub: .\run_tests.ps1
```

Oczekiwany wynik: **59 testów zaliczonych**.

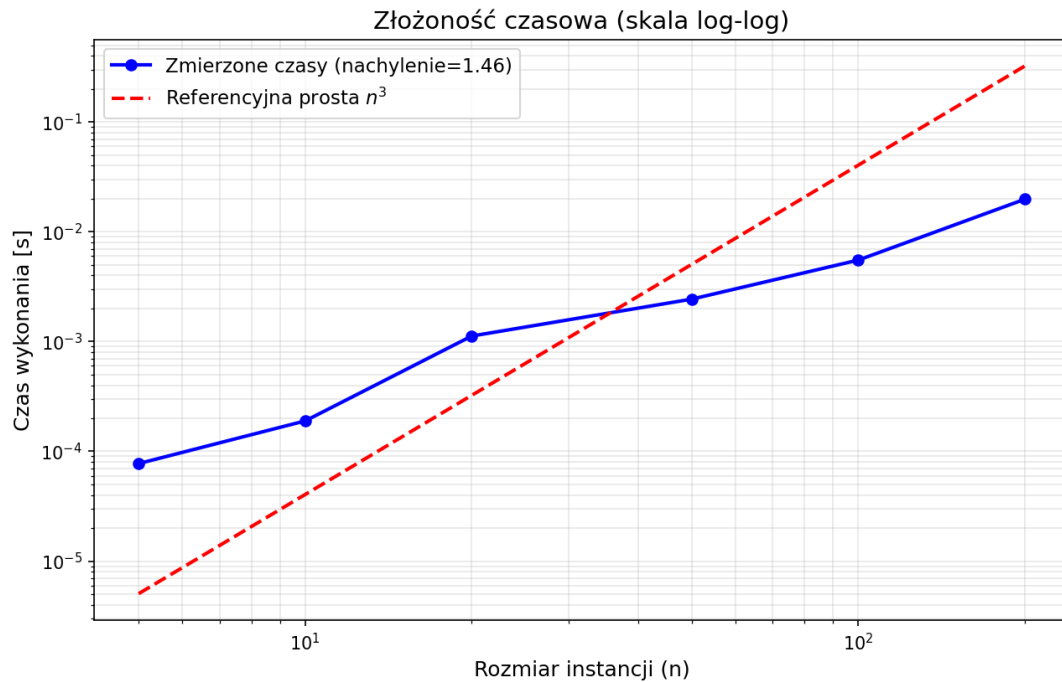
5 Analiza eksperymentalna

5.1 Metodologia

- Instancje euklidesowe (`generator.py`, ziarno 42), po 5 powtórzeń dla $n \in \{5, 10, 20, 50, 100, 200, 500\}$.

- Pomiar czasu: `benchmark.py` → `timing_results.csv`.
- Jakość aproksymacji: porównanie z brute force dla $n \leq 11$ → `quality_results.csv`, `comparison_results.csv`.
- Wykresy: `plots.py` na podstawie plików CSV.

5.2 Złożoność czasowa

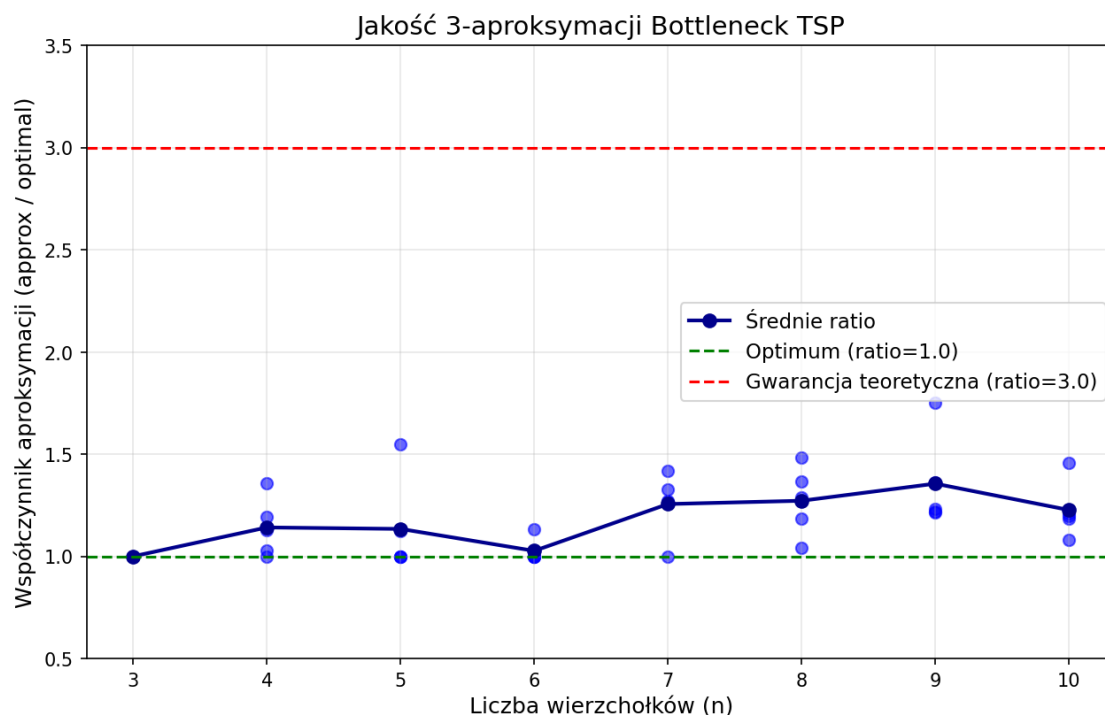


Rysunek 1: Złożoność czasowa solvera w skali log-log dla $n \in \{5, 10, 20, 50, 100, 200, 500\}$. Nachylenie dopasowanej prostej wynosi $\approx 1,64$, co jest zgodne z teoretycznym $O(n^2)$ (i wyraźnie lepsze niż $O(n^3)$, którego linia odniesienia jest zaznaczona przerywanym kolorem czerwonym).

Wnioski — czas:

- Empiryczne nachylenie log-log $\approx 1,64$ leży poniżej teoretycznego 2,0 (Prim na macierzy), co wynika z drobnych narzutów stałych oraz dominacji szybkich operacji NumPy dla mniejszych n — mimo tego algorytm pozostaje w klasie wielomianowej i zachowuje się zgodnie z gwarancją $O(n^2)$.
- Dla praktycznych rozmiarów ($n \leq 500$) czas wykonania nie przekracza ~ 70 ms, co jest w zupełności wystarczające do celów projektu.
- Brute force (porównanie dla $n \leq 11$) rośnie factorialnie i staje się nieużyteczny już przy $n \approx 11$ –12 (sekcja 5.4).

5.3 Jakość aproksymacji (3-aproksymacja)



Rysunek 2: Współczynnik $\text{ratio} = w_{\text{approx}}/w_{\text{opt}}$ w funkcji n dla $n = 3, \dots, 11$ (kropki: pojedyncze instancje, linia: średnia z powtórzeń). Wszystkie punkty mieszczą się znacznie poniżej teoretycznej granicy 3,0 (linia czerwona).

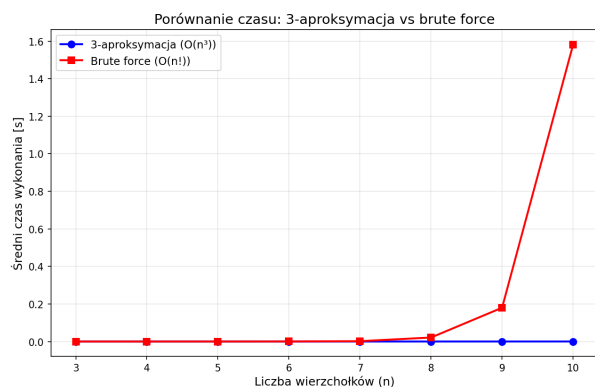
Na instancjach testowych (ziarno 42) uzyskano:

- dla $n = 3$: $\text{ratio} = 1,0$ (algorytm zawsze trafia w optimum — każda permutacja jest cyklem optymalnym),
- dla $n = 5-8$: średnie $\text{ratio} \approx 1,10-1,20$, pojedyncze maksima do $\approx 1,55$ (w jednym przypadku dla $n = 6$ skok do $\approx 1,89$),
- dla $n = 9-11$: średnie $\text{ratio} \approx 1,20-1,32$, maksima nie przekraczają $\approx 1,76$.

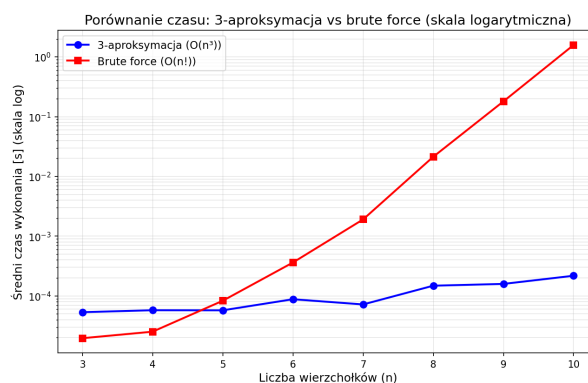
Wnioski — jakość:

- Wszystkie zmierzone ratio spełniają gwarancję teoretyczną ≤ 3 z dużym zapasem.
- Średnie ratio rośnie z n bardzo łagodnie i pozostaje w zakresie $\sim 1,0-1,3$.
- Praktyczna jakość rozwiązania jest **znacznie lepsza** niż pesymistyczna granica dowodu — typowe zachowanie dla aproksymacji TSP.

5.4 Porównanie czasów: aproksymacja vs brute force



(a) Skala liniowa.

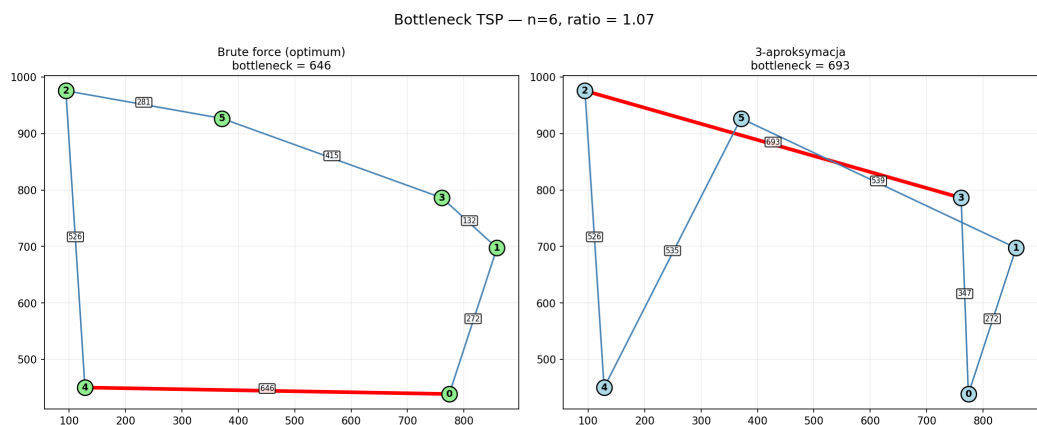


(b) Skala logarytmiczna.

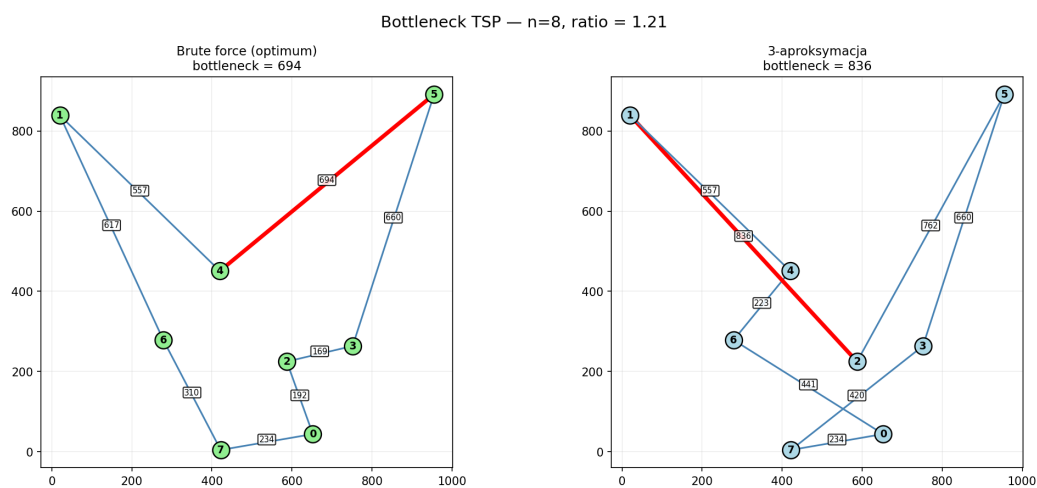
Rysunek 3: Porównanie czasu wykonania 3-aproksymacji i metody brute force dla $n \in \{3, \dots, 11\}$. W skali liniowej widać wybuchowy wzrost kosztu brute force (przy $n = 11$ średnio $\sim 7,3$ s), natomiast w skali logarytmicznej widać, że solver utrzymuje się stale w okolicach 10^{-5} – 10^{-4} s.

Wniosek: Algorytm aproksymacyjny zapewnia rozwiązanie bliskie optimum przy koszcie wielomianowym $O(n^2)$, podczas gdy metoda dokładna ma złożoność silnie wykładniczą (faktorialna liczba permutacji). Już dla $n = 11$ aproksymacja jest około pięć rzędów wielkości szybsza.

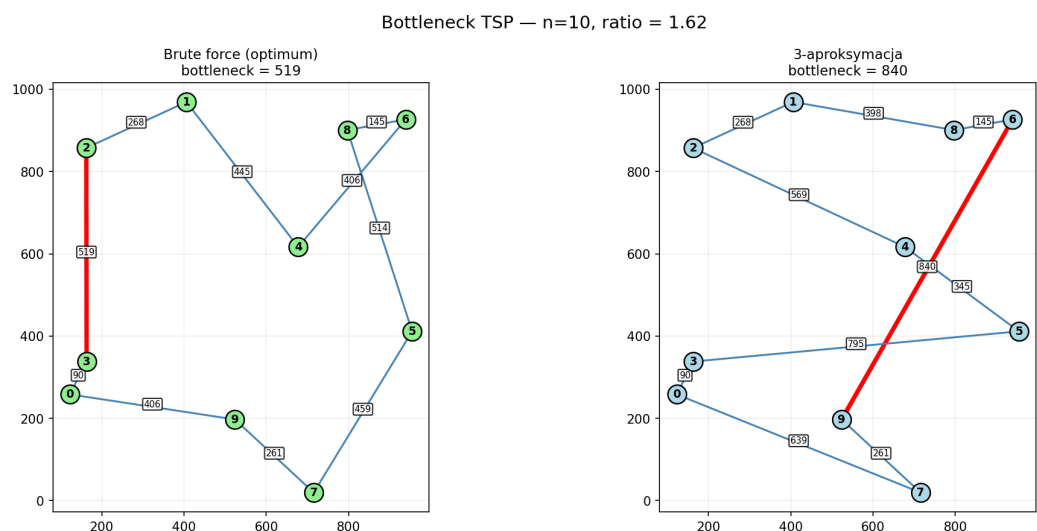
5.5 Wizualizacja tras — porównanie z optimum



(a) $n = 6$, ratio = 1,07.



(b) $n = 8$, ratio = 1,21.

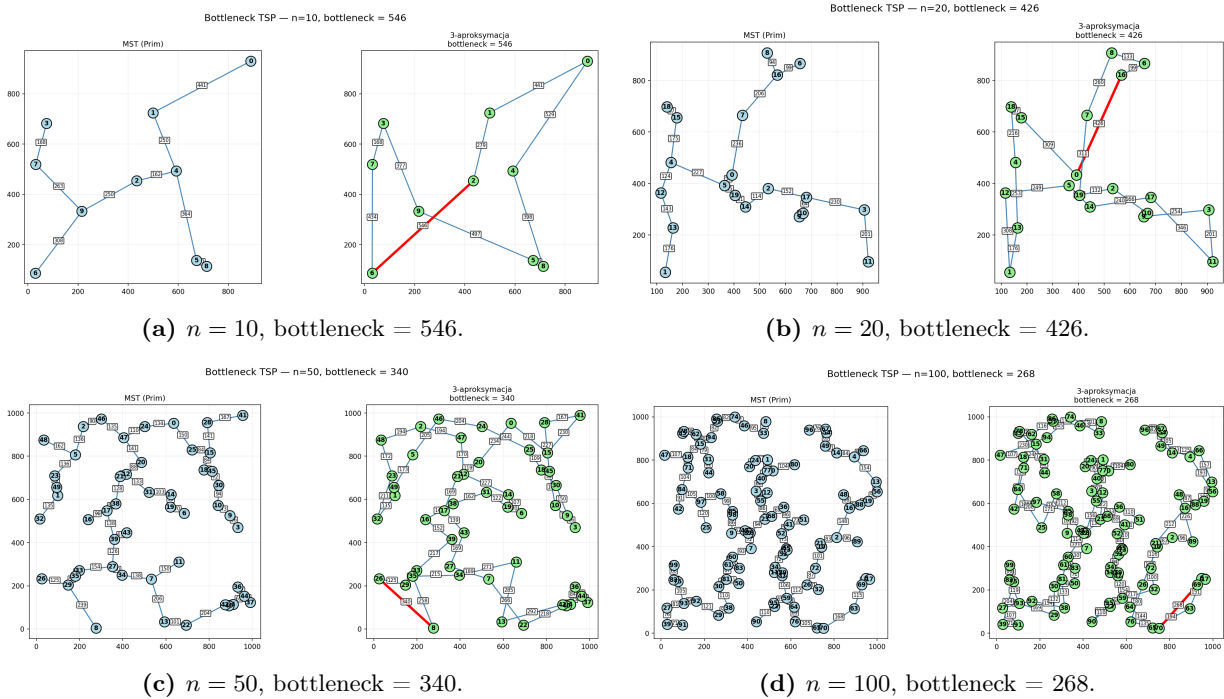


(c) $n = 10$, ratio = 1,62.

Rysunek 4: Porównanie tras: optimum brute force (lewa kolumna) vs 3-aproksymacja (prawa kolumna). Pogrubiona czerwona krawędź zaznacza bottleneck danego rozwiązania.

5.6 Wizualizacja MST oraz cyklu aproksymacji

Dla większych instancji nie istnieje praktycznie obliczalne optimum, ale można zwizualizować strukturę algorytmu: drzewo rozpinające (MST) i cykl wyznaczony przez 3-aproksymację z zaznaczeniem najcięższej krawędzi.



Rysunek 5: Wizualizacja działania algorytmu na większych instancjach: MST (po lewej w każdym panelu) oraz cykl 3-aproksymacji (po prawej, czerwona krawędź = bottleneck).

6 Wnioski końcowe

1. **Poprawność strukturalna:** Solver zwraca poprawny cykl Hamiltona; testy automatyczne (59) przechodzą.
2. **Teoria vs praktyka:** Złożoność empiryczna (nachylenie $\log\text{-}\log \approx 1,64$) pozostaje w klasie wielomianowej i jest zgodna z deklarowanym $O(n^2)$; jakość rozwiązań systematycznie lepsza niż górna granica 3-aproksymacji.
3. **Użyteczność:** Program obsługuje pliki wejściowe, konsolę, wizualizację MST i cyklu, generator instancji oraz pipeline benchmark $\rightarrow$ wykresy.
4. **Ograniczenia:** Brute force tylko dla bardzo małych n (do ~ 11).

7 Podsumowanie testów automatycznych

Pakiet `pytest` weryfikuje m.in.:

- poprawność cyklu Hamiltona i spójność raportowanego bottleneck,
- własności MST (liczba krawędzi, spójność),
- 3-aproksymację dla $n \leq 8$ (brute force w testach),
- pomiary czasu i przybliżone skalowanie kwadratowe (markery `performance`).

8 Bibliografia

- Dokumentacja wstępna projektu (Bottleneck TSP, Algorytmy zaawansowane 2025/2026).

- Johnson, D.S. — *Approximation algorithms for combinatorial problems*, JCSS 1974.
- Repozytorium: <https://github.com/hubsonsss/TSP-Bottleneck>.